

DESIGN AND DEVELOPMENT OF AN INTERACTIVE UNIVERSITY COURSE REGISTRATION PORTAL

Saveetha Institute of Medical and Technical Sciences (SIMATS) PROJECT REPORT

Name: Pravin R
Register Number: 192421292

1. Abstract

This project presents the Interactive University Course Registration Portal, a responsive, browser-based application built to make it easier for students to look through available courses and submit their registration details. The build relies on HTML/XHTML for the page structure, CSS for the visual layout and responsiveness, and JavaScript for handling validation, live course selection, credit totals, and the final registration summary directly in the browser.

The portal's look and feel are built around SIMATS' academic identity, combining a branded header, a navigation bar, a course catalogue, step-by-step registration guidance, eligibility notes, a student registration form, a live selection summary, and contact details. Visually, the design stays fairly restrained and institutional, using a responsive grid system, a flexible nav bar, hover states, consistent spacing and borders, and breakpoints tuned for mobile devices.

One of the core ideas behind the project is that everything related to registration happens on the client side, with no full-page reloads. Course details live in a JavaScript array of objects, and as soon as a student picks a course, JavaScript locates the matching object and works out how many courses have been chosen along with the combined credit total. Before a registration attempt goes through, the browser checks that the required fields — register number, name, email, semester, and at least one course — have all been filled in correctly.

Overall, the project puts core front-end web development concepts into practice and covers the brief given for the assignment: structured content, organized layout, responsive styling, client-side interactivity, on-the-fly calculations, and debugging with the browser's Developer Tools.

1.1 Keywords

HTML, XHTML, CSS, JavaScript, responsive web design, Flexbox, CSS Grid, form validation, DOM manipulation, JavaScript objects, dynamic summary, client-side interaction.

1.2 Project Outcomes

- Build a structured university portal using semantic HTML elements.
- Present at least five academic courses in a tabular layout.
- Put together a responsive, professional-looking interface.
- Validate all registration data before it is processed.
- Work out the course count and total credits on the fly.
- Show a registration summary without reloading the page.
- Use console logging and browser Developer Tools while debugging.

2. Introduction

Course registration is a routine but important part of academic life, where students choose the subjects they will take in a given semester. A good registration interface should present course information clearly, cut down on input mistakes, and give the student immediate feedback. This project sets out to build a lightweight portal that captures these qualities without needing any server-side database.

The portal splits responsibilities across three technologies. HTML handles the content hierarchy and the form controls; CSS is responsible for the visual presentation and how the layout adapts to different screens; JavaScript drives the interactivity and validation. Keeping these concerns separate makes the project easier to maintain, since the content, appearance, and behavior can each be changed independently of the others.

2.1 Problem Statement

A plain static course listing page can show information but cannot check whether a student's inputs make sense or work out a registration summary on its own. On the other hand, a registration form that lacks clear course details and a responsive layout ends up being hard to use. To address both problems, this project brings together a course catalogue, a registration form, validation logic, and a dynamic summary within a single interface.

2.2 Objectives

- Build a semantic, well-organized university course registration page.
- Provide navigation links to the major sections of the portal.
- Display at least five courses along with their code, name, credits, and type.
- Include registration instructions and eligibility requirements.
- Collect register number, student name, email, department, semester, and course choices.
- Apply responsive CSS using Grid/Flexbox together with media queries.
- Validate mandatory fields as well as the email and semester values.
- Store course information as an array of JavaScript objects.
- Calculate the total number of selected courses and credits through a reusable function.
- Display the registration summary dynamically, without a page reload.

3. System Analysis and Requirements

3.1 Functional Requirements

Requirement	Implementation
University information	SIMATS identity, introductory text, contact details, and an external website link
Course catalogue	Six courses presented in an HTML table
Registration form	Five text/select inputs together with multi-course checkboxes

Validation	JavaScript checks the required fields, email format, semester value, and course selection
Dynamic selection	Selected course names and credit values update live in the summary
Credit calculation	Reusable calculateSummary() function
Registration summary	Student and course details shown without a page reload
Debugging	console.log() output used with the browser Developer Tools

3.2 Non-Functional Requirements

- **Usability:** the interface should be simple enough for a student to use without needing help from an administrator.
- **Responsiveness:** the content should stay usable across desktop, tablet, and mobile screens.
- **Maintainability:** HTML, CSS, and JavaScript are kept in separate files.
- **Performance:** all calculations run locally, so a registration preview does not need a server round-trip.
- **Accessibility:** labels are tied to their inputs, semantic sections are used, and status/error areas use suitable live regions.

3.3 Technology Stack

Technology	Purpose
HTML5 / XHTML principles	Page structure, semantic elements, table and form markup
CSS3	Layout, visual design, box model, Grid, responsive media queries
JavaScript	Validation, DOM manipulation, course data and calculations
Google Chrome / Edge	Testing and Developer Tools
VS Code or similar editor	Source-code development

4. System Design

4.1 Architecture

The application is built on a simple three-layer, client-side structure. The HTML layer holds the page content and controls. The CSS layer is responsible for styling the interface and shaping how it responds to different screen sizes. The JavaScript layer reads the values entered into the form, validates them, matches selected checkboxes to the corresponding course objects, works out the totals, and updates the Document Object Model (DOM) accordingly.

No backend or database is used for this assignment. All registration data lives only within the current browser session and is meant purely as an academic demonstration of client-side interaction.

4.2 Page Structure

Section	Main Content
Header	SIMATS branding and navigation
Home	Portal introduction and registration call-to-action
Courses	Course catalogue table
Instructions	Registration steps and eligibility
Registration	Student details and course selection
Live Summary	Selected courses, total courses and credits
Contact/Footer	Institution and contact information

4.3 Data Model

Course details are represented as an array of JavaScript objects, with each object holding four properties: code, name, credits, and type. Using this structure means the same data set can drive both the course table and the course-selection controls, which avoids duplicating information.

A representative object looks like this: `{ code: 'CSE301', name: 'Web Technology', credits: 4, type: 'Core' }`. When a student checks a box, its course code is used to look up the matching object in the course array.

4.4 Process Flow

- The webpage loads and course details are rendered from the JavaScript array.
- The student fills in their registration details.
- The student selects one or more courses.
- JavaScript refreshes the live course and credit totals.
- The student clicks Confirm Registration.
- JavaScript checks that all required values are present and valid.
- If validation fails, an error message appears and the submission is stopped.
- If validation passes, the registration summary is displayed without a page reload.

5. User Interface and Front-End Design

5.1 Visual Design

The interface leans on an institutional color palette — deep green, warm neutral surfaces, and muted gold accents. The hero section is designed to make a strong first impression, while the catalogue and registration areas use clean cards and tables to keep information easy to scan. Typography pairs a display serif for the main headings with a modern sans-serif for controls and body text.

5.2 Navigation

A sticky header keeps the Home, Courses, Registration, and Contact links visible at all times. Internal anchor links let visitors jump straight to different sections of the same page, and an external link points to the official Saveetha website.

5.3 Course Catalogue

Course Code	Course Name	Credits	Type
CSE301	Web Technology	4	Core
CSE302	Database Management Systems	4	Core
CSE303	Computer Networks	3	Core
CSE304	Artificial Intelligence	4	Elective
CSE305	Cloud Computing	3	Elective
CSE306	Cyber Security Fundamentals	3	Elective

5.4 Responsive Design

CSS Grid drives the main hero and registration layouts, while Flexbox handles the navigation bar and component alignment. Media queries collapse the multi-column layouts down to a single column on smaller screens, and the form fields and course options also switch to a single-column arrangement once the viewport gets narrow.

5.5 CSS Features Used

- Element, class, ID, pseudo-class, and descendant selectors.
- Padding, margin, border, width, height, and box-sizing for the box model.
- CSS Grid for the major page layouts.
- Flexbox for the navigation bar and component alignment.
- Hover and focus transitions on interactive elements.
- A sticky header along with responsive media queries.

6. JavaScript Implementation

6.1 Course Data

The course catalogue is kept in a JavaScript array called `courses`, where each entry is an object holding the course code, name, credit value, and course type. The interface builds the HTML table rows and the course-selection controls dynamically from this array.

6.2 Validation

The `validateForm()` function checks that every mandatory field has been filled in. It also checks the register-number pattern, the format of the student's name, the email pattern, whether the semester value is in range, and whether at least one course has been picked. If any of these checks fail, the form submission is blocked and a visible error message is shown.

6.3 Dynamic Calculation

The reusable `calculateSummary(selectedCourses)` function takes in the selected course objects. The number of objects passed in gives the course count, and a `reduce()` operation adds up their credit values to produce the total credits. The function then writes these results into the relevant summary elements in the DOM.

As an example, choosing Web Technology (4 credits), Artificial Intelligence (4 credits), and Cloud Computing (3 credits) results in 3 selected courses and 11 total credits.

6.4 DOM Interaction

The page listens for checkbox change events and input events. Whenever one of these fires, `updatePreview()` reads the current form values and the selected courses and refreshes the live summary. The submit event calls `event.preventDefault()` so that the browser does not reload the page.

6.5 Debugging with Developer Tools

Browser Developer Tools were used throughout development. `console.log()` statements are included to print out the selected course data, the calculated totals, any validation issues, and the details of a successful registration. This makes it possible to use the Console to confirm that the JavaScript data flow is behaving as expected.

7. Testing and Results

7.1 Test Cases

Test Case	Input / Action	Expected Result
Empty form	Click Register	Error: complete mandatory fields
Invalid register number	Short/invalid format	Registration rejected
Invalid email	student@	Email validation error
Invalid semester	Outside 3–8	Semester validation error

No course	All course boxes unchecked	Course-selection error
One course	Select one course	Course count = 1; credits match course
Multiple courses	Select 3 courses	Names and total credits update
Valid registration	All valid fields + courses	Success summary shown
Responsive view	Resize to mobile width	Columns stack and remain usable
Debugging	Open F12 Console	Course and validation logs visible

7.2 Sample Successful Result

Summary Field	Sample Value
Student Name	Sample Student
Register Number	24CSE101
Department	Computer Science & Engineering
Semester	Semester 5
Selected Courses	Web Technology; Artificial Intelligence; Cloud Computing
Total Courses	3
Total Credits	11

7.3 Result Discussion

The finished portal meets the client-side requirements set out for it. Course details are laid out in a table, the form gathers all the required student information, and inputs that do not pass validation are caught before registration can proceed. Course selection and credit calculation both happen instantly in the browser, and the final summary is built dynamically without needing a full page refresh.

Because the layout is responsive, the same application adapts comfortably to smaller screens. The project also shows how console output can support debugging, making it possible to check the values being used inside the JavaScript functions while the app is being built.

8. Conclusion and Future Enhancements

8.1 Conclusion

The Interactive University Course Registration Portal shows how the core building blocks of web development can come together to form a complete, responsive, interactive academic tool. HTML gives the page a meaningful structure along with functional form controls; CSS turns that structure into a polished, responsive interface; and JavaScript layers on validation, data handling, calculations, and dynamic updates to the DOM.

The project satisfies the assignment brief by including university information, navigation, a six-course table, lists and hyperlinks, a full registration form, responsive CSS, browser-side validation, JavaScript course objects, a reusable credit-calculation function, a summary that appears without a page reload, and debugging carried out through Developer Tools.

8.2 Future Enhancements

- Connect the interface to a backend service and database so registrations persist.
- Add student authentication using institutional credentials.
- Prevent duplicate course registrations and enforce timetable-conflict checks.
- Add prerequisite checks and credit-limit validation.
- Generate a downloadable registration receipt or PDF.
- Provide an administrator dashboard for viewing and managing registrations.
- Add accessibility testing and support for multiple languages.

8.3 Project Folder Structure

The project is organized into separate front-end resources:

File	Description
index.html	Semantic page structure, navigation, course table, form and summary
style.css	Professional visual design, layout, responsive behavior and interactions
script.js	Course data, validation, calculations, dynamic rendering and debugging
portal-image.png	Uploaded visual used in the portal hero area

References

- HTML and CSS concepts drawn from Web Technologies Unit I.
- JavaScript concepts drawn from Web Technologies Unit II.
- Browser Developer Tools and the JavaScript Console, used for debugging.
- Official Saveetha / SIMATS website, used as an institutional reference:
<https://www.saveetha.com/>

Appendix – Requirement Mapping

Question	Requirement	Evidence in Project
1	University title, navigation, courses, contact	Header, Home, Courses, Contact and Footer sections
2	At least five courses	Six courses in the HTML table
3	Lists and hyperlinks	Registration steps, eligibility list, internal and external links
4	Registration form	Register No., Name, Email, Department, Semester and Course Selection
5	CSS and responsive interface	Selectors, box model, Grid, Flexbox, media queries, hover/focus effects
6	JavaScript validation	validateForm() checks required data, email and semester
7	Course object/array	courses array contains course objects
8	Reusable calculation	calculateSummary(selectedCourses)
9	Dynamic summary	DOM updates with student and selected-course information
10	Developer Tools	console.log() used for validation, selection and registration debugging